



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

Machine Learning & Deep Learning

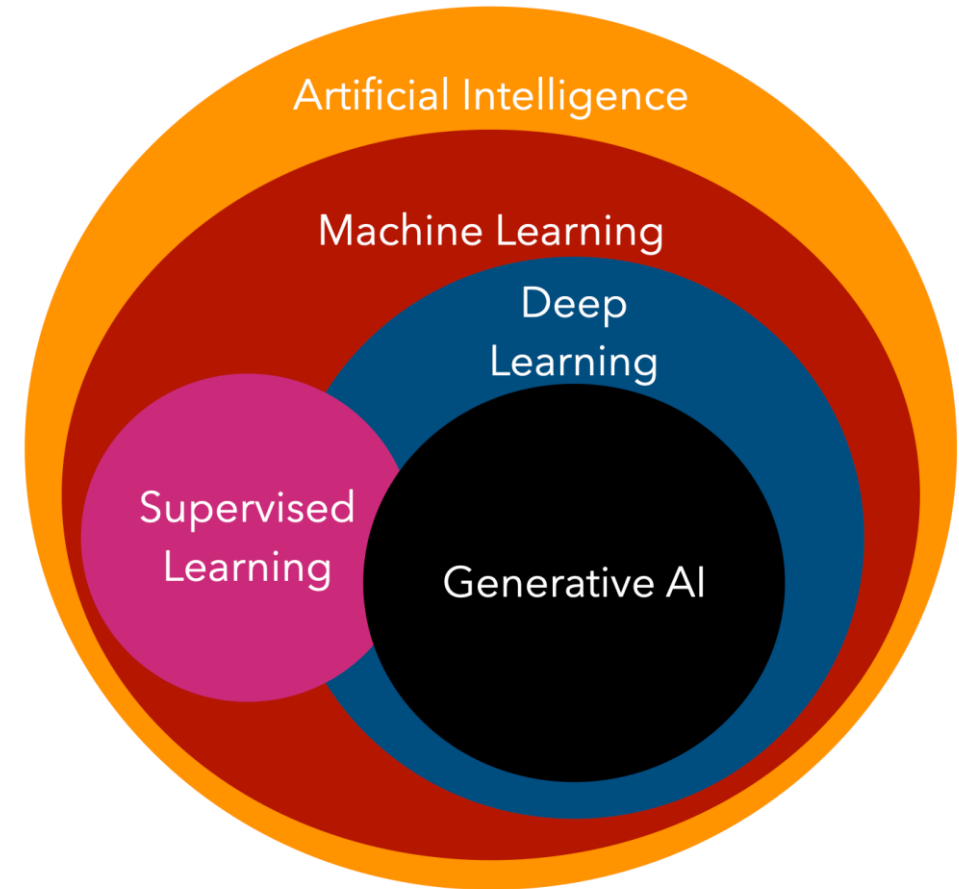
GENERATIVE ARTIFICIAL INTELLIGENCE

2. Juni 2026



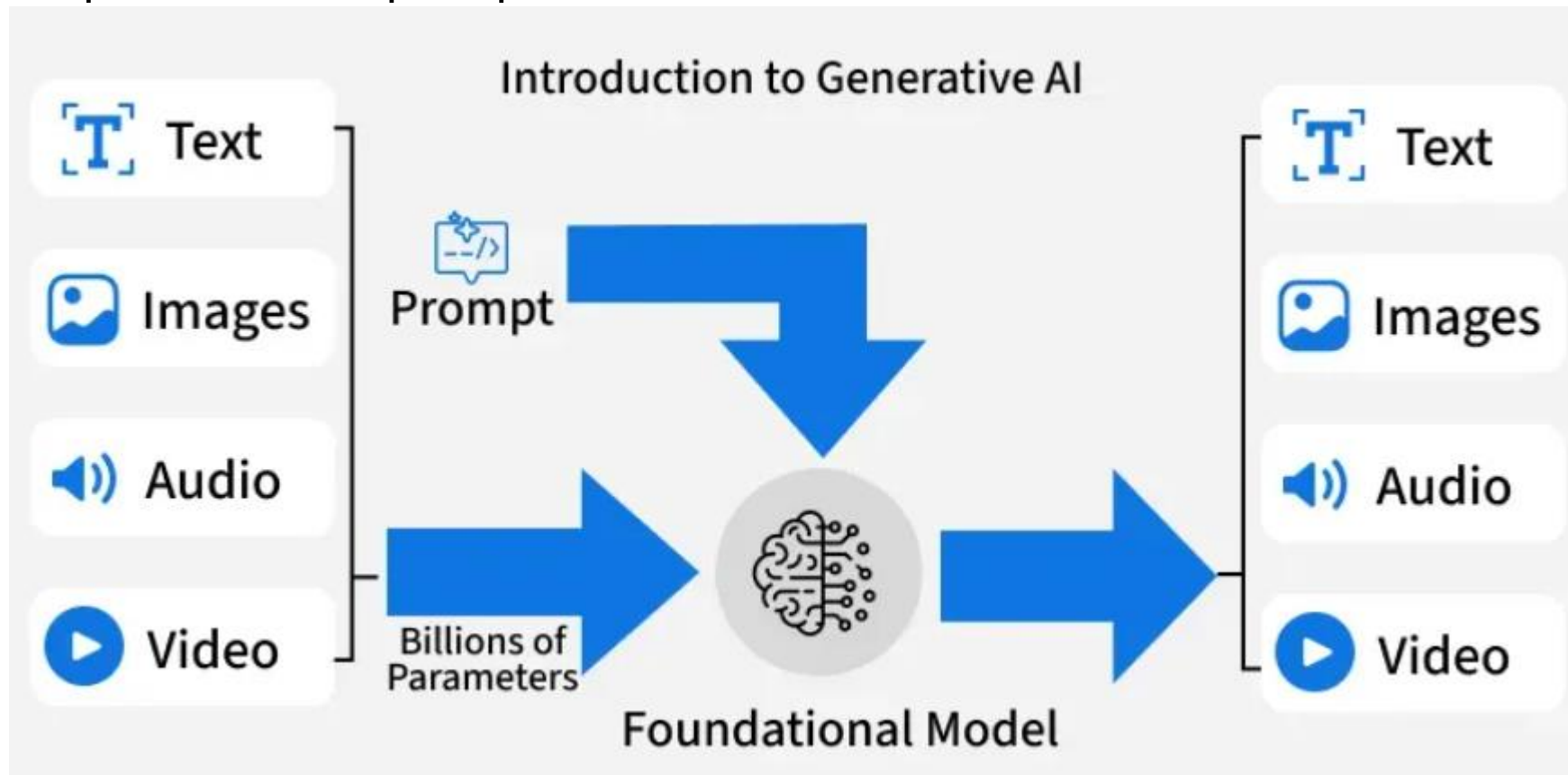
What ist Generative AI ?

- Creates content based on what it has learned from existing content
- Subset of deep learning
- Processes labeled and unlabeled data:
 - supervised
 - unsupervised
 - semi-supervised

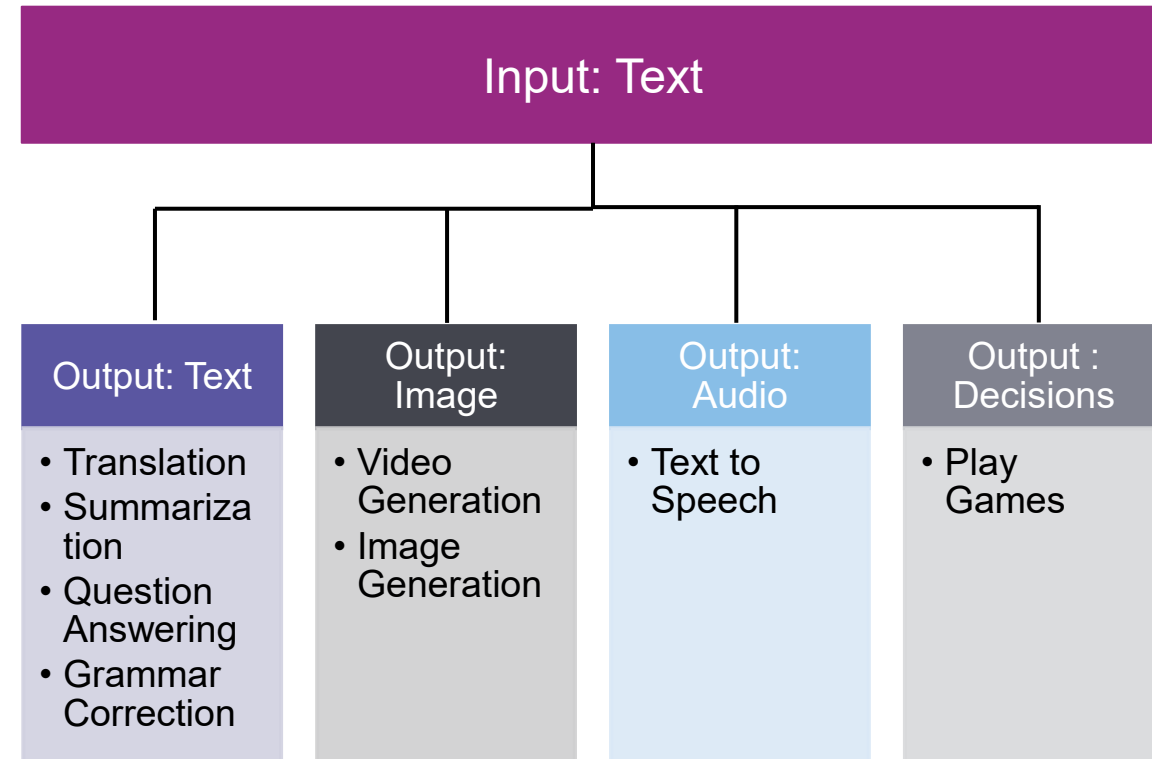
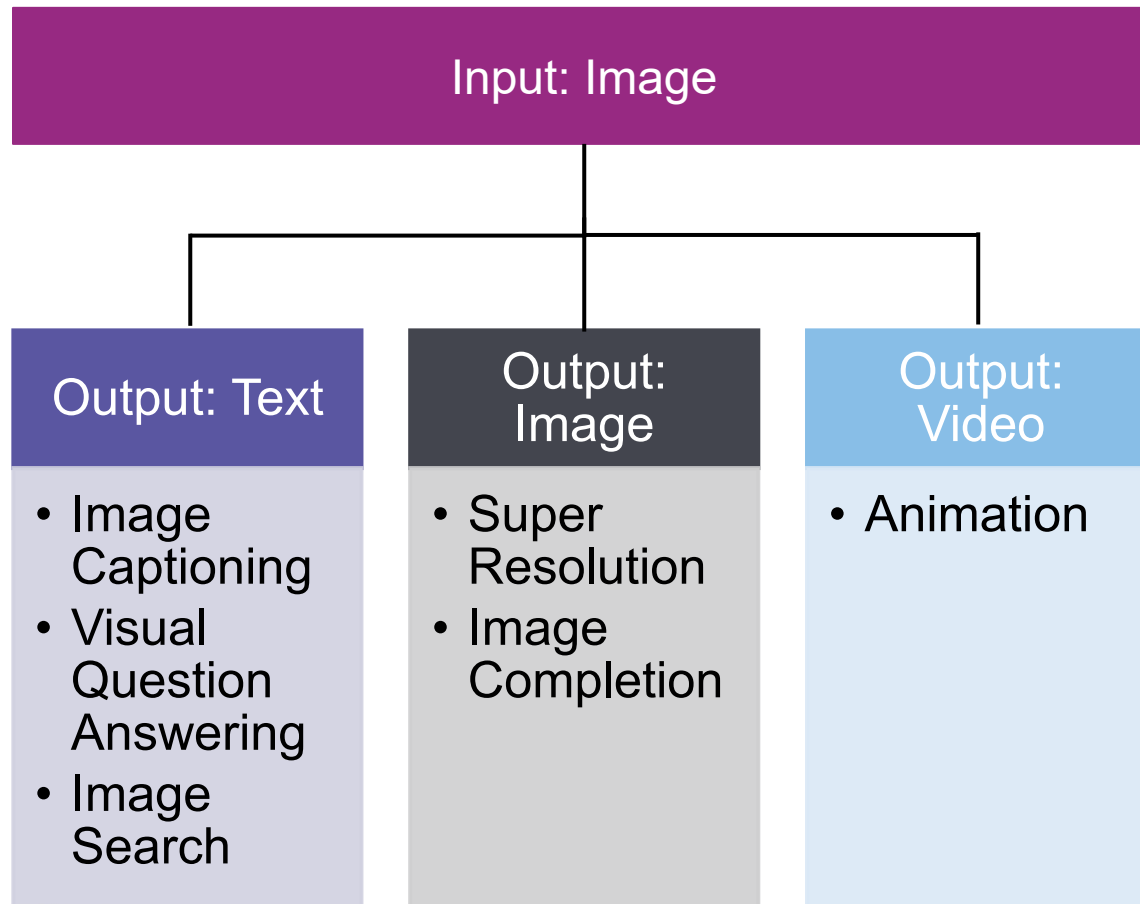


What is Generative AI ?

GenAI creates new content like text, images, audio, and code based on the data it was trained on. Model, which understands patterns in existing data and then generates novel outputs in response to user prompts.



What ist Generative AI ?



Autoregressive Models

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_{<t})$$



- The output is generated one step at a time, where each new step depends on all previously generated steps.
- They learn a probability distribution.
- At each timestep t , the model predicts the next element x_t using everything it has already produced
- The model behaves like a storyteller:
 - Sees beginning of story
 - Decides what comes next and adds it
 - Uses updated story to predict next part

Autoregressive Models

Greedy Decoding

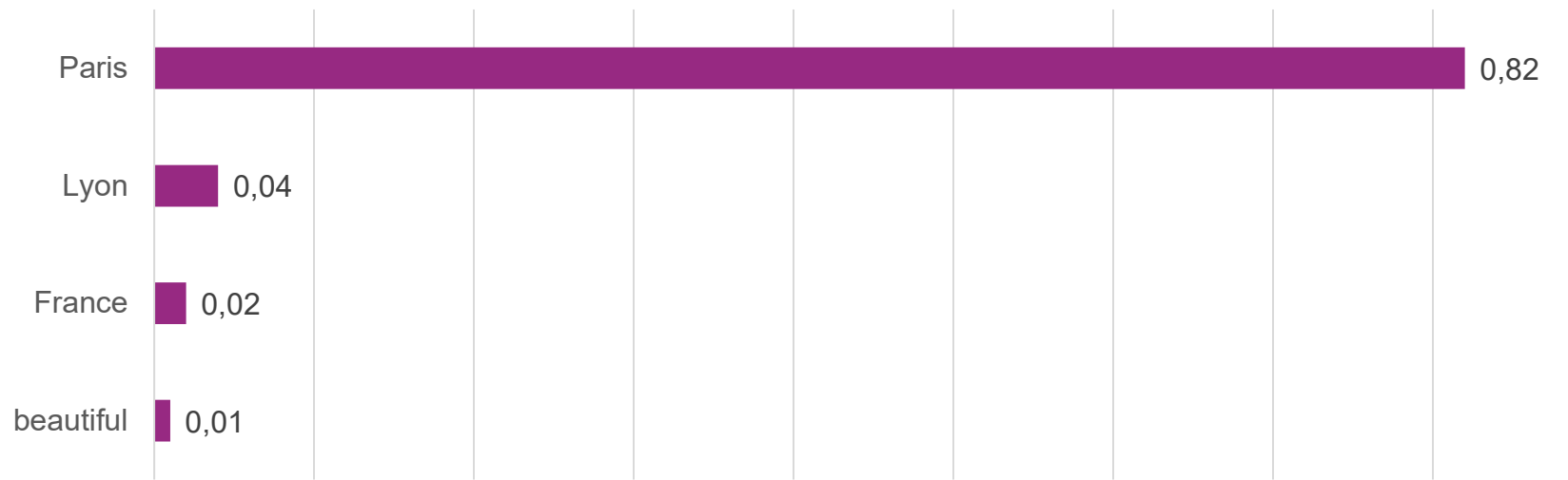
- Always choose the most likely next token.
- Deterministic, fast, but often repetitive

Temperature

- Rescale token probabilities before sampling.
- Controls randomness: low = safe, high = creative.

Top-k

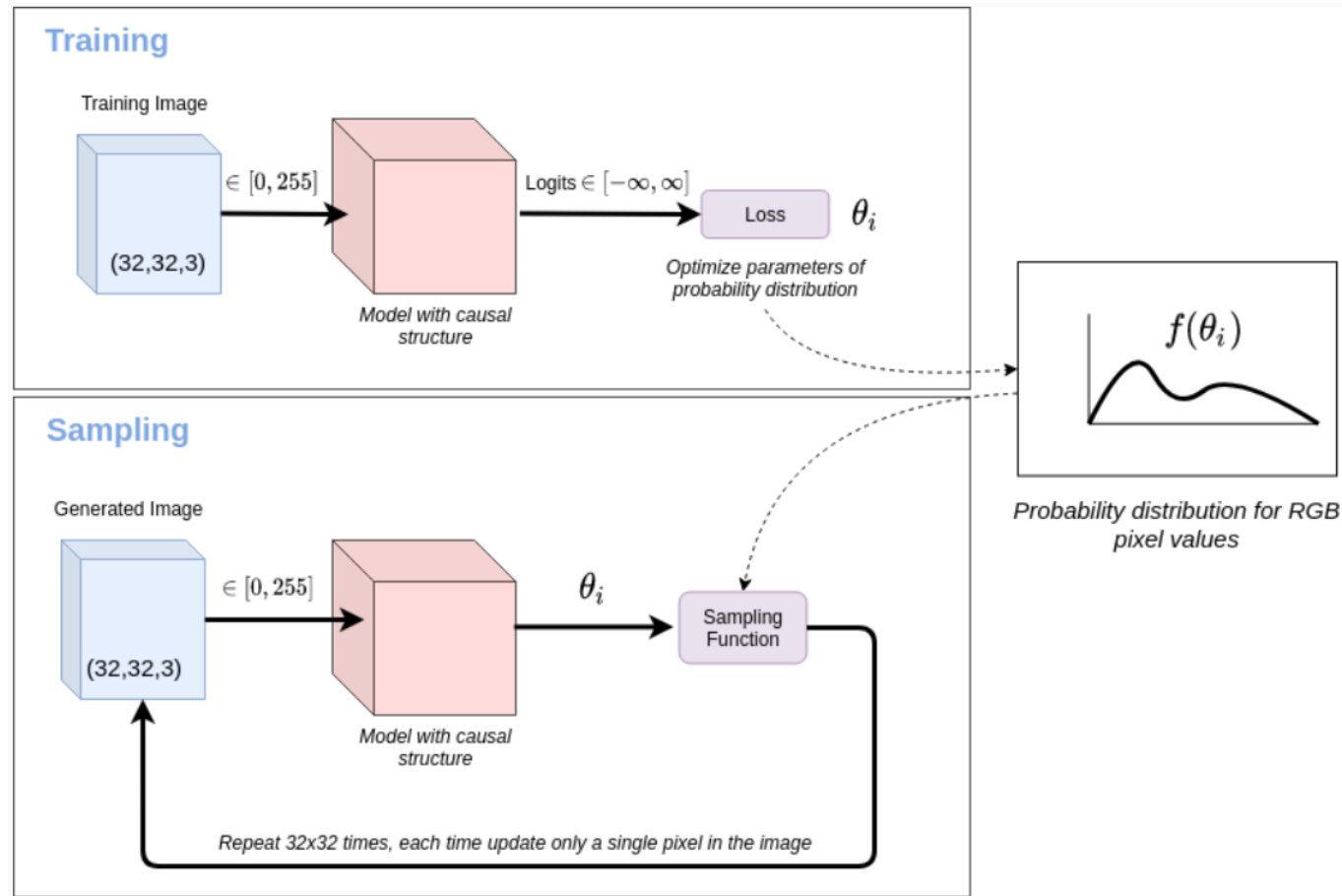
- Sample only from the k most likely next tokens
- Avoids very unlikely tokens, keeps some diversity



Autoregressive Models

- Those steps are then repeated until a stopping criterion is met.
- Such as:
 - End-of-sequence token approaches (Token at the end of the input examples, which can be predicted as usual word to stop the prediction; default for LLMs)
 - Maximum length limit (max. Tokens/Length to prevent infinite loops)
 - Probability Drop-Off (probabilities for all next tokens fall below a threshold; rare behavior)

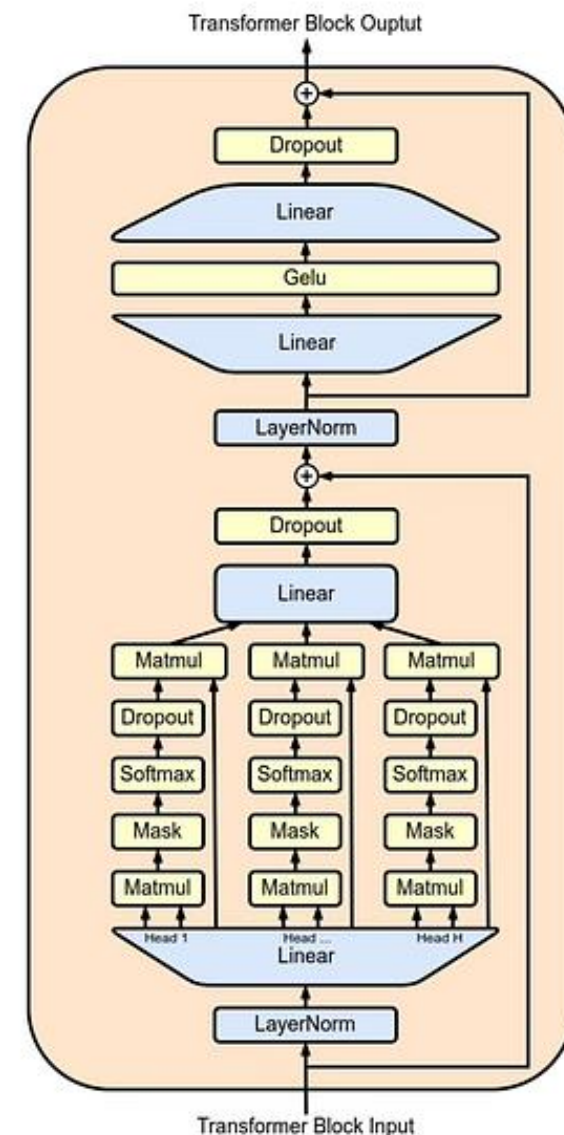
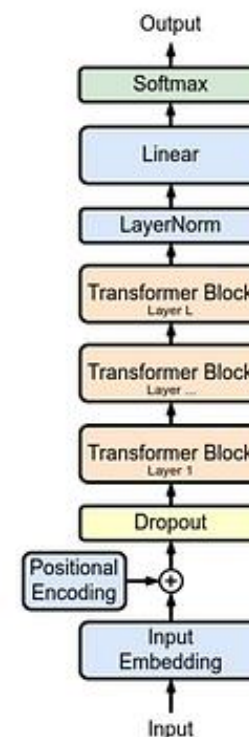
Autoregressive Models



Autoregressive Models

Generative Pre-trained Transformer (GPT)

- Build upon transformer architecture
 - Only on the Decoder
- GPT treats the prompt as the start of a sequence and continues generating.
- Why Does GPT Work So Well?
 - Scale - A very large number of parameters and a massive training dataset.
 - Self-attention - Direct relationships between tokens within the context.
 - Next-token prediction as a universal training signal - Virtually any text provides training examples.
 - Instruction tuning / human feedback - The raw language model is shaped into a helpful assistant.



Evolution of GPT

Phase	Model / Version	Year	Main Development	Key Capabilities
Early Development	GPT	2018	Introduced pre-training followed by fine-tuning for language tasks	Learned basic language patterns and could be adapted to downstream NLP tasks
Early Development	GPT-2	2019	Scaled up the model size and training data	Improved text generation, coherence, and fluency
Scaling Phase	GPT-3	2020	Major increase in scale and performance	Few-shot and zero-shot learning; stronger general language abilities
Advanced Capabilities	GPT-4	2023	Improved reasoning, context understanding, and multimodal capabilities	Better performance on complex tasks, stronger reliability, image/text understanding
Recent Improvements	GPT-4.1 / GPT-4.5	2025	Better coding, longer context handling, and improved instruction following	More useful for programming, document understanding, and complex user instructions
Current Generation	GPT-5	2026	More advanced and efficient system balancing speed and deep reasoning	Improved accuracy, fewer errors, better complex-task understanding, stronger coding and reasoning performance

Autoregressive Models

- Strengths:
 - Good sequence generation quality
 - Simple/interpretable probabilistic structure
 - Supports multiple domains (text, image, audio)
- Weaknesses:
 - Slow inference (token by token generation)
 - Error accumulation (early wrong prediction leads to repetitive errors)
 - Difficulty modeling long-term dependencies

Variational Autoencoders

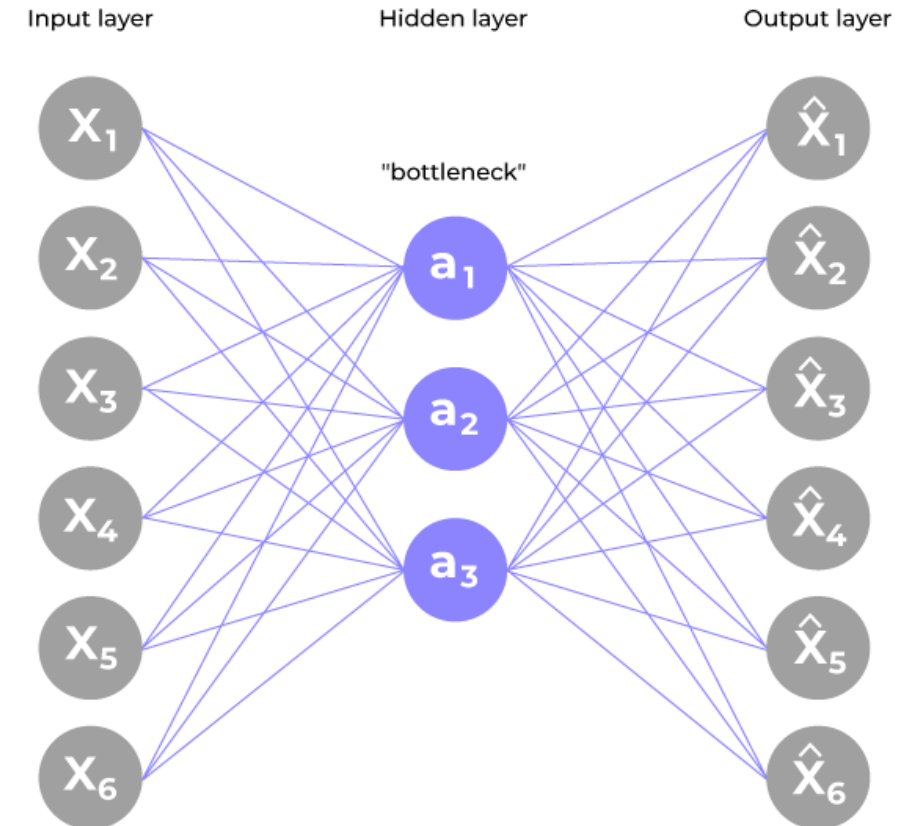
“A variational autoencoder (VAE) is a type of neural network that learns to reproduce its input, and also map data to latent space.”

- VAEs learn to compress data into continuous latent space and then reconstruct new data by sampling from that space.
- Unlike standard autoencoder, which compresses to a fixed code, VAE learns distribution over latent codes (means & variance)
- Model can generate novel, realistic samples and not just reconstruct inputs.

Variational Autoencoders

Autoencoder

- Autoencoders are neural networks, which learn efficient encodings of data based of two main components:
- Encoder: Compresses the input into a dense representation (latent space)
- Decoder: Reconstructs the data from the latent space representation
- By minimizing the reconstruction error, autoencoders learn to represent the data in a latent space while preserving its essential characteristics.



Variational Autoencoders

- VAEs extend autoencoders by introducing probabilistic elements into the latent space.
- Do not learn a deterministic mapping, they learn a distribution over the latent space instead.
- Ability to generate new, diverse samples by sampling from learned distribution.
- Encoder: Outputs parameters of probability distribution (mean and variance). Not a fixed latent vector.
- Decoder: Reconstructs data from samples drawn from the latent distribution.

Variational Autoencoders

Encoder

- Given input data x (e.g., an image):

$$x \rightarrow \text{Encoder} \rightarrow [\mu(x), \sigma(x)]$$

- The mean and standard deviation vector define a gaussian distribution in the latent space:

$$q(z|x) = \mathcal{N}(z; \mu(x), \sigma(x)^2 I)$$

- Model learns continuous latent space where nearby points correspond to similar reconstructions
- The model then samples from the distribution:

$$z = \mu(x) + \sigma(x) \cdot \varepsilon$$

Variational Autoencoders

Decoder

Learns to map latent samples to original data dimensionality:

$$z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

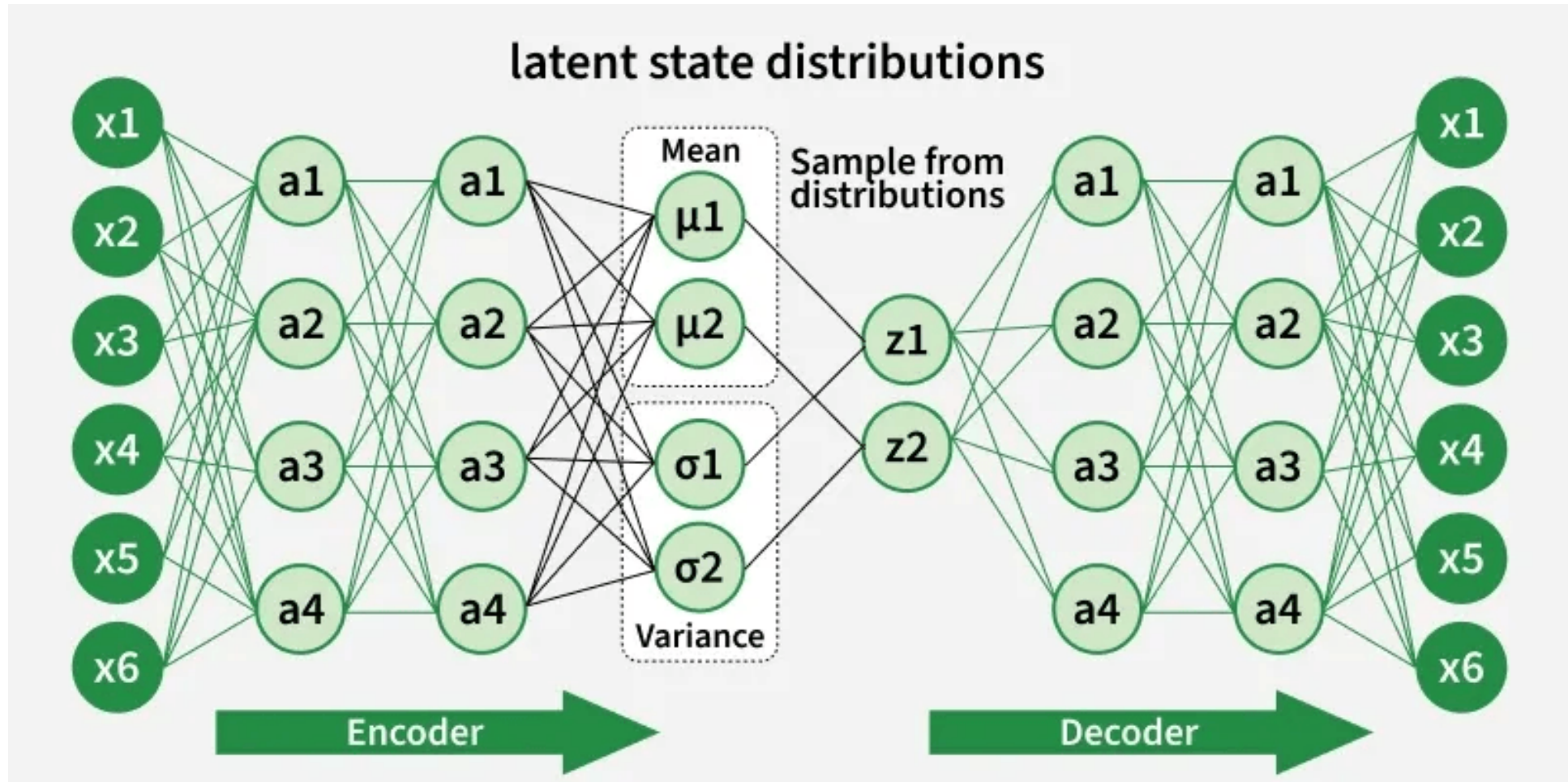
Learns data distribution

Maps abstract latent features to concrete data

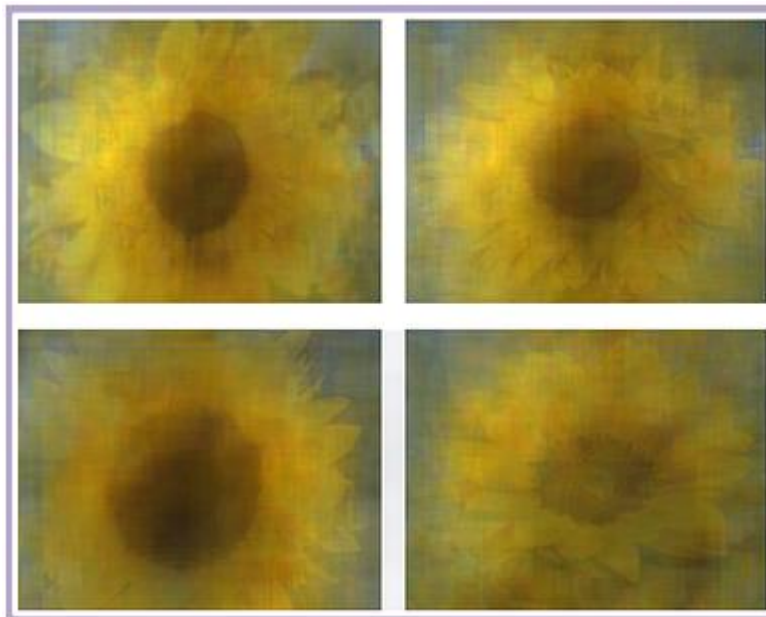
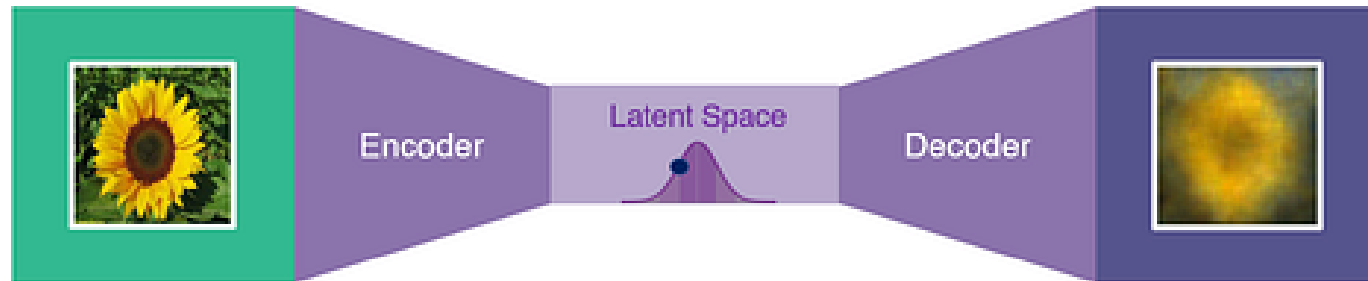
Turns new latent samples into new data

Input $\rightarrow$ compressed distribution $\rightarrow$ sample $\rightarrow$ reconstruction

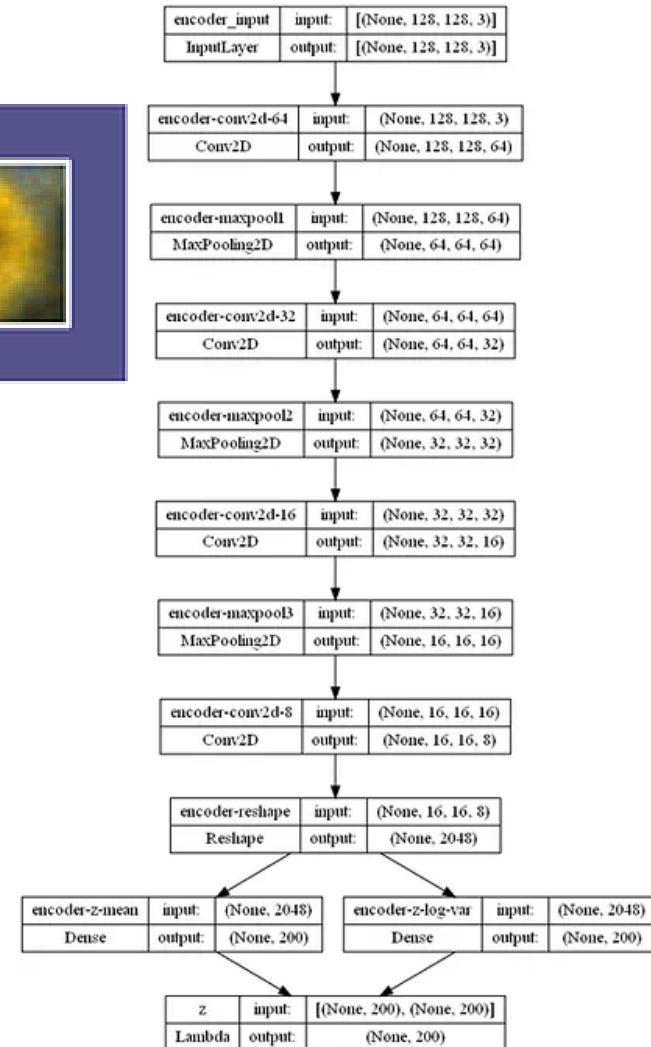
Variational Autoencoders



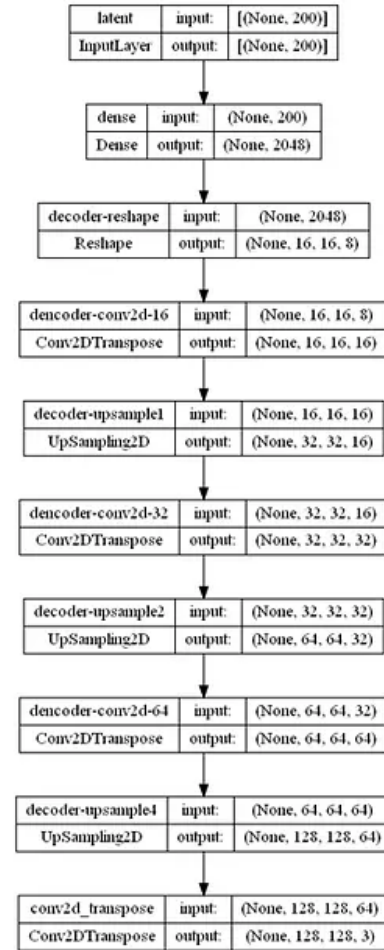
Variational Autoencoders



Encoder



Decoder



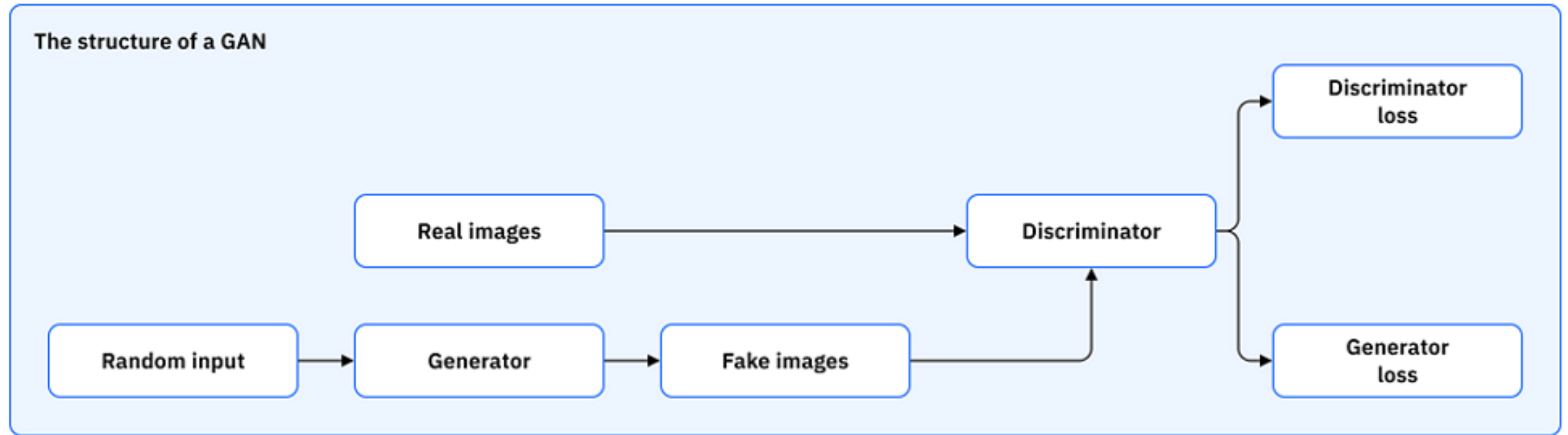
Variational Autoencoders

- Strengths:
 - Stable and reliable training by optimizing a single objective
 - Smooth and interpretable latent space
 - Enables interpolation, controlled generation
 - Probabilistic modeling capability
 - Allows sampling, uncertainty estimation
- Weaknesses:
 - Generated images tend to be blurry (due to pixel-wise reconstruction losses, which encourage averaged outputs)
 - Limited ability to capture fine details or complex data distributions

Generative Adversarial Networks (GAN)

- GANs are a class of generative models that learn to produce new data samples that are statistically and perceptually similar to samples from a real dataset.
- Instead of explicitly computing the probability distribution of the input data, a GAN trains two neural networks: a generator, which creates fake samples, and a discriminator, which tries to distinguish real samples from fake ones.
- The generator learns to map samples from a latent noise distribution to the data distribution. After training, realistic new data can be generated by sampling noise and passing it through the generator.

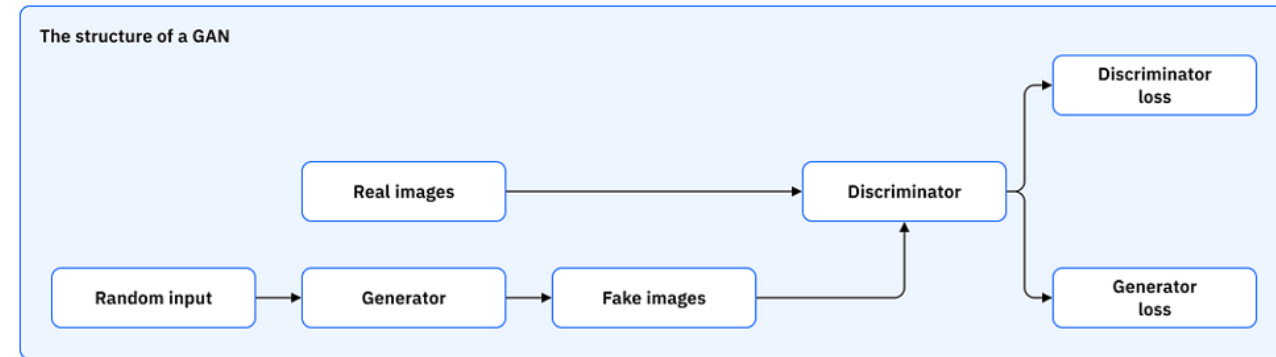
Generative Adversarial Networks (GAN)



Generative Adversarial Networks (GAN)

Generator Model

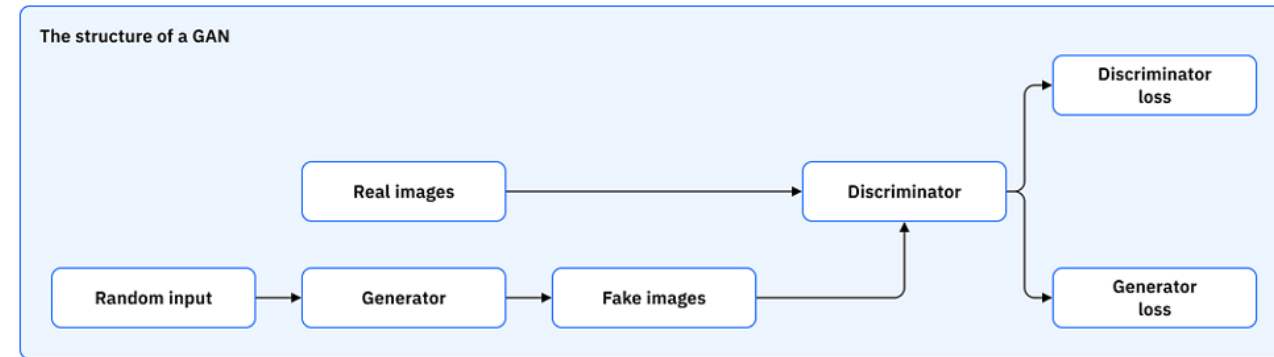
- deep neural network
- generates realistic data samples out of random noise input (vector)
- tries to minimize the generator loss
- goal:
 - create samples that are as realistic as possible and indistinguishable from the real samples.



Generative Adversarial Networks (GAN)

Discriminator

- (convolutional) neural network with binary classifier
- distinguishes real data from generated samples
- tries to minimize the discriminator loss
- goal:
 - improve detection of fake data and when comparing with real samples



Generative Adversarial Networks (GAN)

- GAN loss function (minmax loss):

$$\min_G \max_D \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

- Discriminator tries to:
 - Maximize $\log(D(x))$
 - Maximize $\log(1 - D(G(z)))$
 - Wants to become as good as possible in detecting real vs. fake.
- Generator tries to:
- Cannot influence real-data term $\log(D(x))$, so it tries to:
 - $\min_G \mathbb{E}_z[\log(1 - D(G(z)))]$
- Produces fake samples that make discriminator believe they are real
- Push $D(G(z))$ as high as possible (close to 1)
- Wants fake images to look real enough that D is fooled.

Generative Adversarial Networks (GAN)

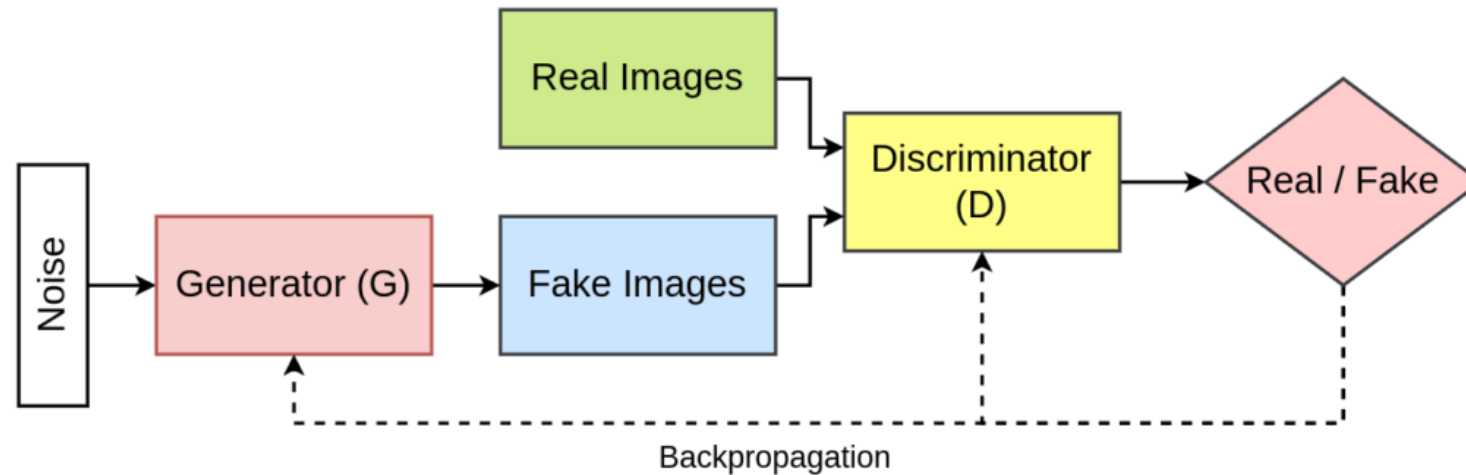
What happens when training succeeds?

- The generator effectively approximates the true data distribution. So, the output distribution becomes very close to the real data distribution.
- The discriminator becomes less able to distinguish real from fake. Ideally classification accuracy is around 50%, meaning fake samples are nearly indistinguishable from real ones by the discriminator.
- New novel realistic samples can now be generated based on sampling new noise vectors.
- Very good for data generation, data augmentation, superresolution,

GAN Types

Vanilla GAN

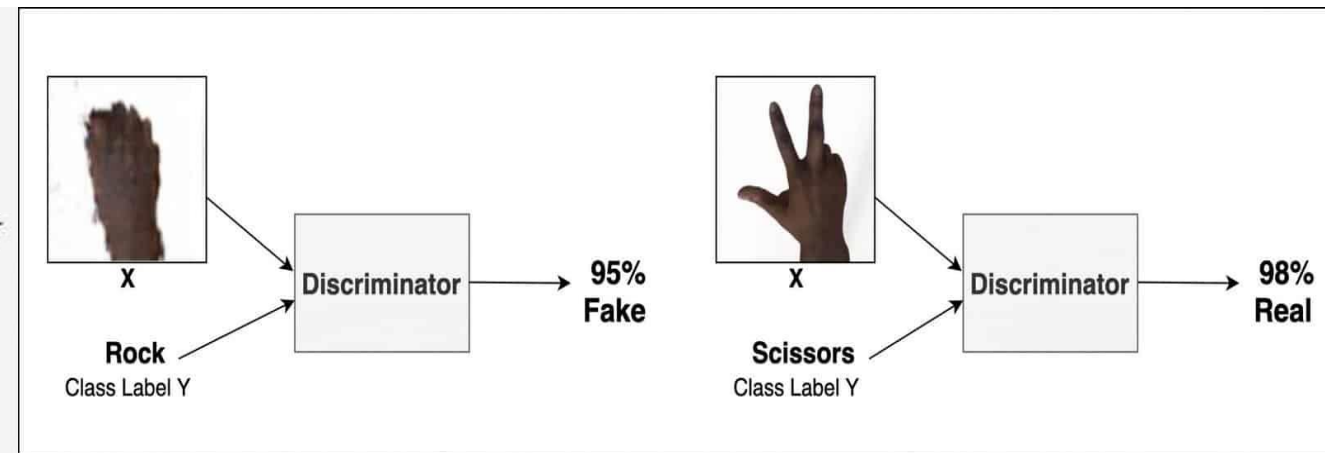
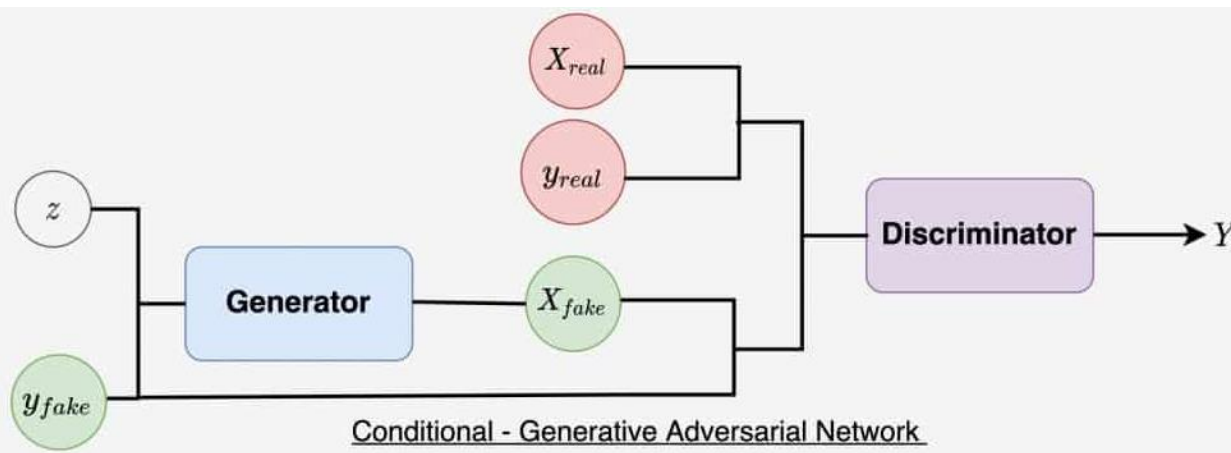
- Vanilla GAN is the simplest type of GAN.
- It consists of:
 - A generator and a discriminator both are built using multi-layer perceptrons (MLPs).
 - The model optimizes its mathematical formulation using stochastic gradient descent (SGD).
 - Can suffer from unstable training and limited output variety



GAN Types

Conditional GAN (CGAN)

- A conditional variable (y) is fed into both the generator and the discriminator.
- This ensures that the generator creates data corresponding to the given condition (e.g. generating images of specific objects).
- The discriminator also receives the labels to help distinguish between real and fake data.



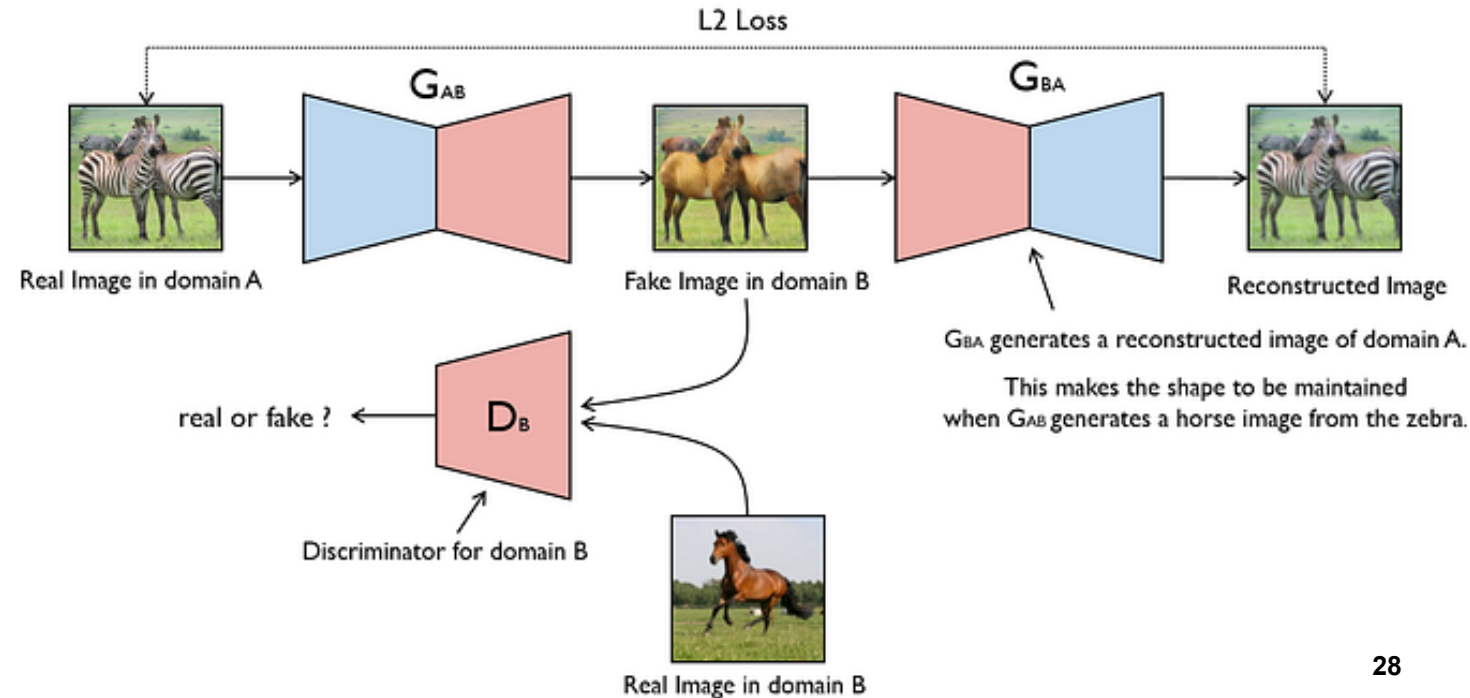
GAN Types

CycleGan

- Translates images between two domains without paired examples
- Uses two generators and two discriminators
- Key idea: cycle consistency

$$x \rightarrow G(x) \rightarrow F(G(x)) \approx x$$

- Combines adversarial loss with cycle consistency loss



Generative Adversarial Networks (GAN)

Strengths:

- Produce extremely sharp, high-fidelity images (often more realistic than VAEs)
- Learn complex, high-dimensional data distributions
- Very fast inference (generation = single forward pass through generator)

Weaknesses:

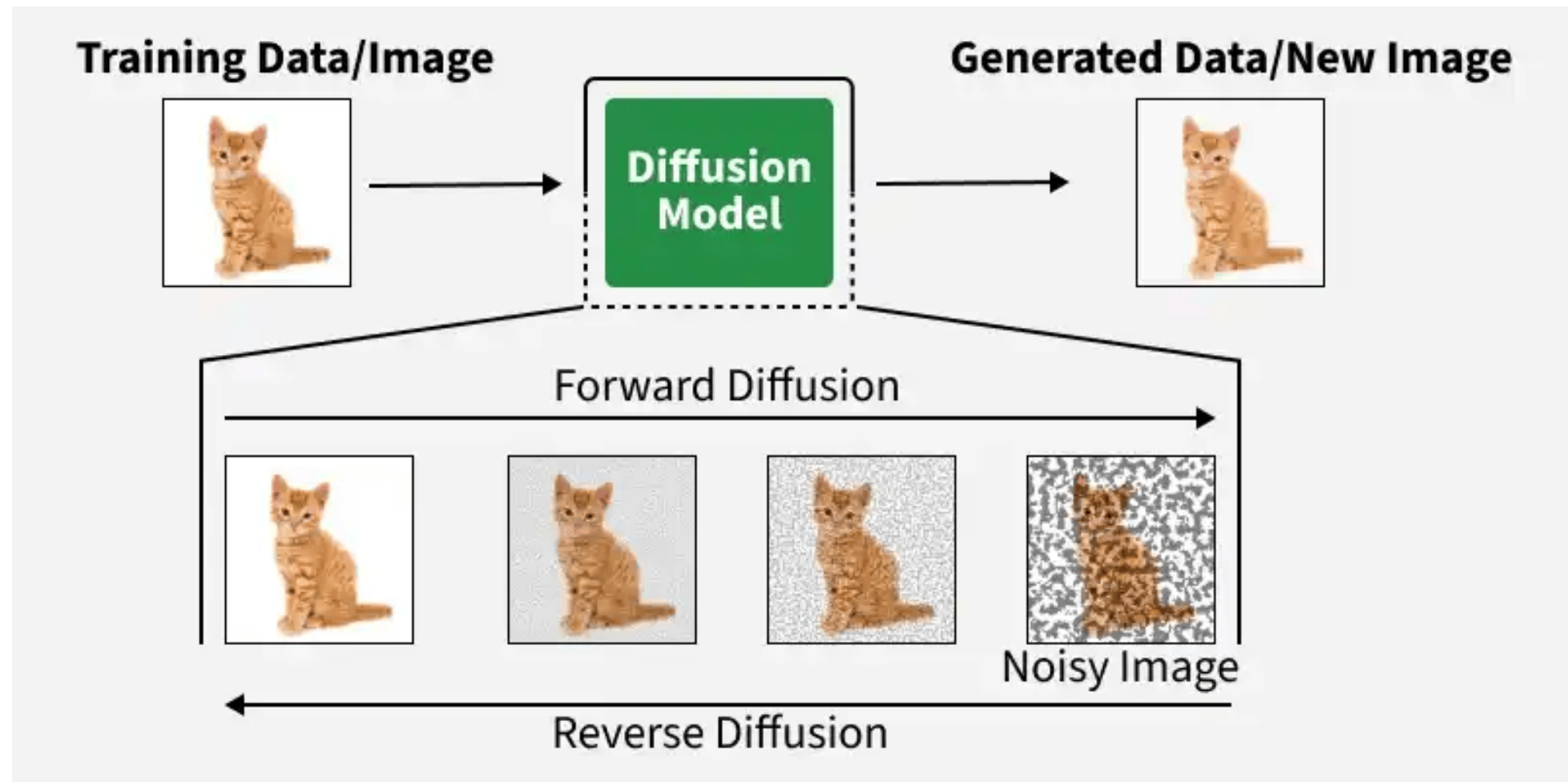
- Training is unstable (generator and discriminator need balance to avoid collapse)
- Mode collapses easily (generator produces only few repeating samples instead of covering full data distribution)
- Sensitive to hyperparameters and architecture design
- Often needing large datasets for good performance

Diffusion Models

Class of generative models that create new data by starting with pure noise and gradually denoising it into a structured output.

- Idea:
 - Forward Process: Gradually corrupt real data by adding noise over many tiny steps until it becomes random noise
 - Reverse Process: Learn how to invert this corruption step-by-step, until data becomes a clean, realistic sample
- The neural network learns the reverse process.

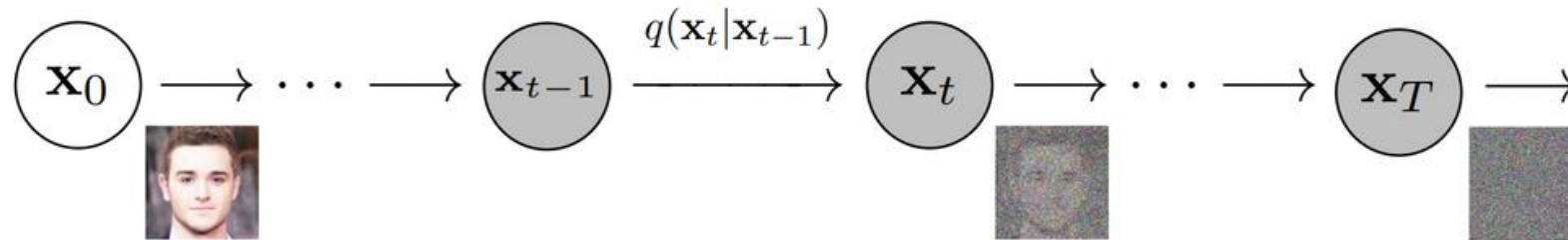
Diffusion Models



Diffusion Models

Forward Diffusion Process (Noise Addition):

- Fixed, non-learned Markov chain (next state only depends on the current state)
- At each timestep t , a small amount of Gaussian noise is added
- After many steps, data becomes essentially pure Gaussian noise

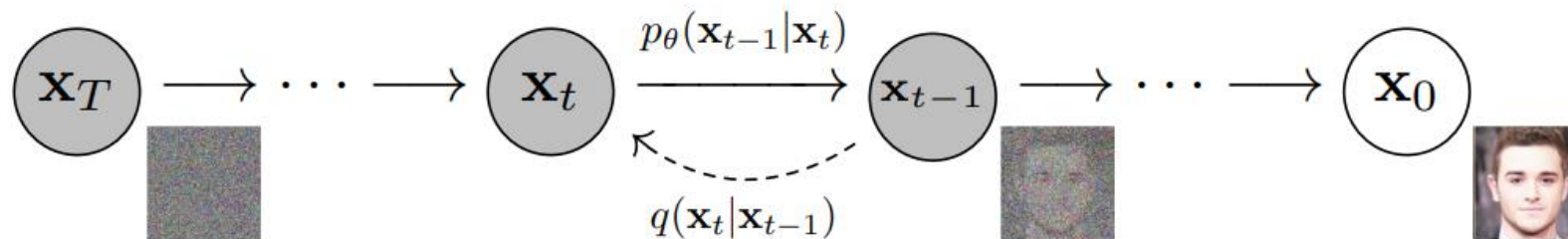


- The process:
 - Is mathematically simple, stable
 - Provides paired examples (noisy image, clean image) for training

Diffusion Models

Reverse Diffusion Process (Denoising)

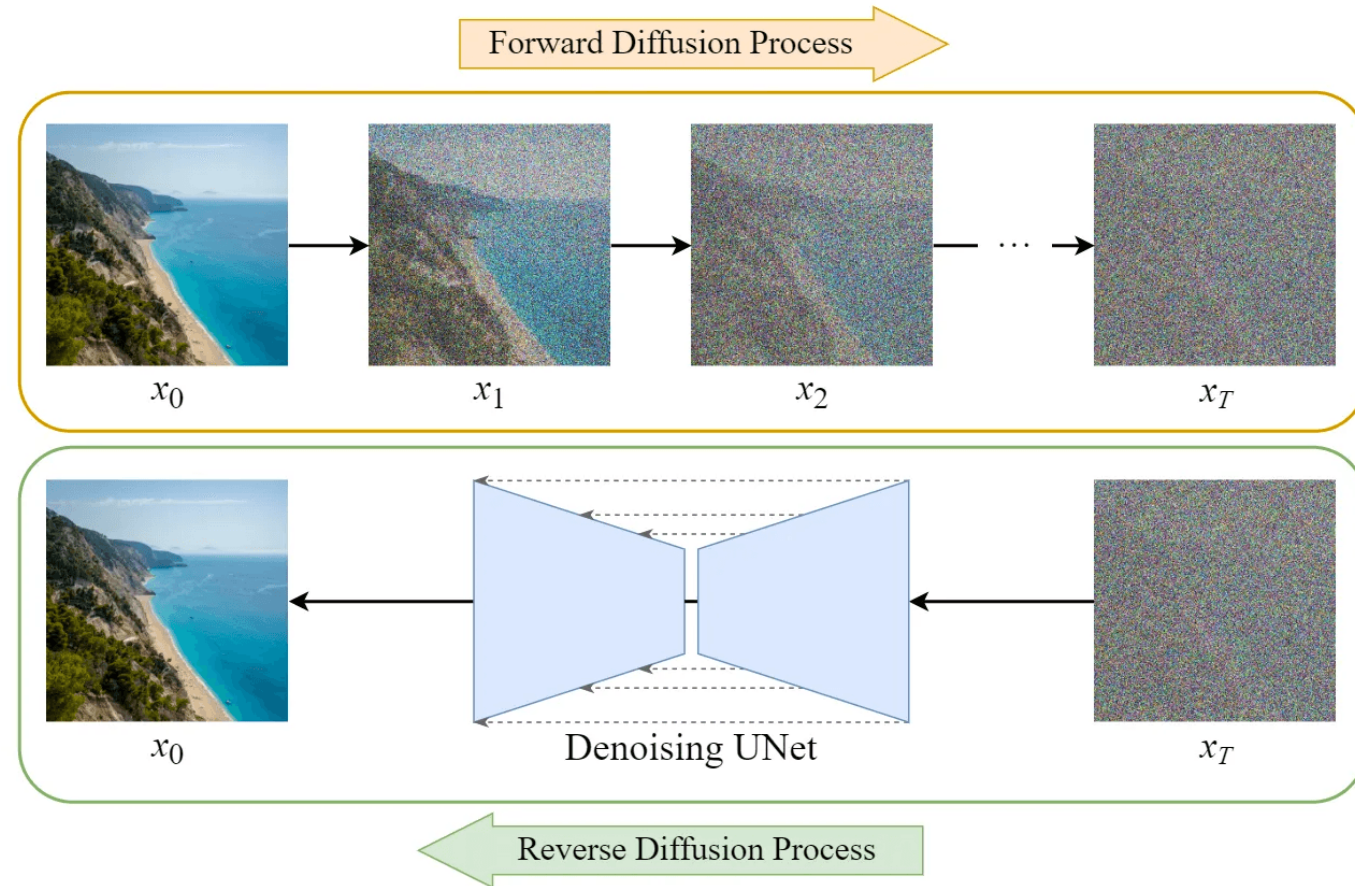
- The generative model learns to reverse the forward pass
- Given a noisy sample x_t the model predicts:
 - The noise that was added
 - The clean input



The training objective of the diffusion model is to minimize the difference between the true noise added in the forward process and the noise predicted by the neural network.

Diffusion Models

Stable Diffusion



Diffusion Models

Strengths:

- High fidelity. Detailed, diverse outputs.
- Stable training. Easy objective and predictable convergence.
- Works for multiple modalities.

Weaknesses:

- Slow generation. Multiple steps required for denoising.
- Expensive to train at scale.
- Complex sampling procedures. Many variants and hyperparameters.

Models Overview

Model	Idea	Example
Autoregressive	Generate one token/pixel at a time	GPT, PixelCNN
Diffusion	Start from noise and iteratively denoise	Stable Diffusion, DALL·E-style image generation
Variational Autoencoders	Encode data into latent space, then decode samples	NVAE (Nouveau VAE), CVAE (Conditional VAE)
Generative Adversarial Network	Generator learns to fool a discriminator	StyleGAN, CycleGAN

Evaluation

Evaluating GenAI is challenging because outputs are open-ended, subjective, and multimodal. Common evaluation goals include:

- Quality: Are outputs coherent, fluent, realistic?
- Diversity: Does the model avoid repetition and mode collapse?
- Accuracy: Does it stay grounded in facts?
- Safety: Are outputs harmless?
- User Preference: Do humans prefer the model's outputs?

Evaluation

Overview

Modality	Common Metrics	What They Mainly Capture
Text	BLEU, ROUGE, METEOR, BERTScore, Perplexity	reference similarity, fluency, semantic overlap
Image	FID, Inception Score, CLIPScore, LPIPS, SSIM/PSNR	realism, diversity, prompt alignment, reconstruction quality
Code	Pass@k, Unit Test Pass Rate, Static Analysis	functional correctness, bugs, efficiency
Audio	MOS, WER, PESQ/STOI, Speaker Similarity	naturalness, intelligibility, signal quality
Video	FVD, Temporal Consistency, CLIP-based Alignment	realism, motion stability, prompt alignment

Evaluation

Text

Metric	Goal	Evaluation	Limitation
BLEU	Measures n-gram overlap with reference text	High score if many word sequences match	Penalizes valid paraphrases
ROUGE	Measures word/phrase overlap with reference summary	Checks whether reference content appears in output	Does not fully capture meaning or quality
METEOR	Uses overlap, stemming and synonyms	More flexible than BLEU	Still reference-dependent
BERTScore	Compares contextual embeddings	Captures semantic similarity	Does not guarantee factual correctness
Perplexity	Measures how well a model predicts text	Lower = less surprising to the model	Low perplexity $\neq$ useful or truthful

Evaluation

Image

Metric	Goal	Evaluation	Limitation
FID	Compares real and generated image feature distributions	Lower FID = generated images look more like real images	Dataset-level metric, not for single images
Inception Score	Measures recognizability and diversity	Good images are classifiable and varied	Does not measure prompt alignment
CLIPScore	Compares image and text prompt embeddings	Higher score = image semantically matches prompt	Can miss fine details
LPIPS	Measures perceptual similarity between two images	Lower = images look similar to humans	Mostly useful for reconstruction
SSIM / PSNR	Compare image to reference at pixel/structure level	High score = close to reference image	Often weak for open-ended generation

Evaluation

Audio

Metric	Goal	Evaluation	Limitation
MOS	Humans rate audio quality or naturalness, usually from 1–5.	Average human opinion about how natural the audio sounds.	Expensive, slow, and subjective.
WER	Measures word-level transcription errors.	Lower WER means the speech content is easier to understand correctly.	Measures intelligibility, not naturalness or expressiveness.
PESQ / STOI	Compare generated or enhanced speech to a reference signal.	Estimate speech quality and intelligibility from signal properties.	Better for speech enhancement than open-ended audio generation.
Speaker Similarity	Measures whether generated speech sounds like a target speaker.	Compares speaker embeddings of target and generated audio.	Useful but ethically sensitive because of deepfake risks.

Evaluation

Video

Metric	Goal	Evaluation	Limitation
FVD	Compares feature distributions of real and generated videos.	Like FID for images, but includes temporal video features.	Dataset-level metric; not enough for single videos or prompt correctness.
Temporal Consistency	Measures stability of objects, identities, colors, and motion over frames.	A good video should not flicker or randomly change important details.	Difficult to capture with one universal metric.
CLIP-based Alignment	Measures semantic alignment between video and text prompt.	Checks whether frames or video embeddings match the prompt.	Can miss causality, exact actions, order, or fine-grained details.
Human Preference	Humans rate, compare, or rank generated videos.	Captures realism, aesthetics, plausibility, and overall usefulness.	Expensive and subjective.

Key Takeaways

- Generative AI learns patterns in existing data to create new content such as text, images, audio, video, or code.
- Autoregressive Models generate outputs step by step; each new token, pixel, or sample depends on previous ones.
- GPT use decoder-only Transformers to predict the next token and generate text autoregressively.
- VAEs learn a probabilistic latent space and generate new samples by decoding latent representations.
- GANs train a generator and discriminator in competition to produce realistic samples.
- Diffusion Models start from noise and gradually denoise it into realistic data.
- Evaluation requires multiple metrics because outputs are open-ended, subjective, and modality-specific.

Control Questions

Why can a generative model produce outputs it has never seen exactly during training?

Why is conditional generation $p(x|c)$ important for practical GenAI applications?

What is the role of latent space in a VAE?

In a GAN, what happens if the discriminator becomes too strong too early?

What is the main difference between how GANs and diffusion models generate images?

Why is the forward diffusion process fixed, while the reverse process is learned?

Why is human evaluation still important in GenAI?

Give an example where an automatic metric looks good but the output is unacceptable.